

## CBC Teachers' Toolkit - Project Process Report

### 1. Project Context and Problem Definition

This project was built to solve a practical challenge in the Ndejje University community context: student-teachers and teachers need fast, reliable, and curriculum-aligned teaching support that works in low-connectivity environments.

The application targets CBC teaching support by offering:

- Strictly Offline-First bundled resources (PDFs/Videos)
- Resource-level favorites management
- Notes and topic favorites
- Scheme of Work builder
- External system viewer integration for all media types

Why this step was necessary:

- The capstone requires a real community-relevant problem, not a generic demo.
- Defining a clear user problem early guided all architecture and feature choices.

### 2. Requirements Analysis and Scope Definition

The implementation was aligned with the capstone brief:

- Kotlin + Jetpack Compose
- MVVM architecture
- LazyColumn/LazyRow dynamic lists
- Navigation across multiple screens
- Room persistence
- Unit/integration testing
- MainActivity as entry point only

Scope decisions:

- Enforce strictly offline-first architecture by disabling remote sync (GitHub/Firebase).
- Bundle all resources in `assets/Resources/` for guaranteed access.
- Use system `Intent.ACTION\_VIEW` for media (PDF/Video) instead of internal players to reduce technical debt.
- Implement resource-level bookmarking (`isFavorite`) in `TeachingResourceEntity`.

Why this step was necessary:

- It ensured grading rubric compliance and prevented scope drift under tight deadlines.

### 3. Architecture Design

The app follows a layered design:

- UI layer: Compose screens
- State layer: `SubjectViewModel`
- Data layer: `TopicRepository`
- Persistence: Room database with DAO + entities

Key entities:

- TopicEntity
- FavoriteEntity
- NoteEntity
- TeachingResourceEntity (with `isFavorite` status)
- SchemeOfWorkEntity

Why this step was necessary

- Separating concerns improves maintainability, testability, and easier feature growth.
- MVVM and repository patterns are explicitly expected in the project brief.

### 4. Navigation and Screen Planning

Implemented screens;

- Splash
- Home
- Subjects
- Resource Detail
- Updates
- Library
- About
- Scheme Builder

Why this step was necessary

- Navigation Compose provides a clear user flow and satisfies the "3+ screens" requirement.
- Dedicated screens keep each feature focused and easier to present.

## 5. Offline-First Data and Media Strategy

The app utilizes a strictly offline approach. All media (PDFs, Videos) are bundled within the APK assets.

- Data seeding: Initial content is seeded locally via Room.
- Asset Lifecycle: Resources are copied from `assets/Resources/` to the app's cache directory and opened via `FileProvider` when requested.
- Remote synchronization features (SyncWorker) are disabled to prevent reliance on external connectivity.

Why this step was necessary

- Ensures teachers in zero-connectivity areas have immediate access to all materials.
- Simplifies app maintenance by removing network dependency.

## 6. External Media Integration

To provide a robust viewing experience without increasing app complexity:

- Internal PDF and Video players were removed.
- All media is opened via `DownloadUtils.openDownloadedFile` which triggers a system `Intent.ACTION\_VIEW`.
- This leverages high-quality external viewers already installed on the user's device.

Why this step was necessary:

- Reduces technical debt and app size.
- Guarantees compatibility with various media formats.

## 7. Resource-Level Favoriting

A new favorite system allows users to bookmark specific PDFs and videos independently of topics.

- Updated `TopicDao` to support resource favorite toggling.
- Added a "Bookmarked Materials" section in the Library screen.

Why this step was necessary:

- Provides more granular control for teachers to organize their most-used files.

A dedicated builder allows:

- Week-based planning
- Curriculum-aligned objectives and activities
- Assessment and resources mapping
- Offline save and sharing
- In-app guide explaining how to design a proper scheme

Why this step was necessary:

- This directly answers teacher workflow needs beyond content viewing
- Converts app value from reader to teaching productivity tool

## 8. UI/UX and Navigation Prioritization

- The "About" section was moved to the bottom of the Home screen and side drawer, acting as a secondary utility.
- Primary actions like "Library" and "Scheme Builder" are prioritized in the UI.
- Bookmark icons were added to resource rows in the Detail screen.

## 9. Code Quality and MainActivity Compliance

MainActivity is intentionally minimal

- setContent is CbcToolkitAppRoot()

All business logic and state are outside MainActivity

Previews were added to improve design-time validation

Why this step was necessary

- Directly matches the audit requirement in the capstone brief
- Reduces coupling and accidental logic leakage into Activity

## 10. Testing and Verification

Executed checks

- Unit tests like testDebugUnitTest
- APK build like assembleDebug

Tests cover core behavior such as

- topic observation
- favorites toggling
- note persistence behavior

Why this step was necessary:

- Confirms reliability of key teacher workflows.
- Required by the testing component of the capstone rubric.

## 11. Git Workflow and Incremental Delivery

Work was delivered in small, descriptive commits by feature stage

- data model
- UI modules
- sync behavior
- tests
- docs and polish

Why this step was necessary:

- Improves traceability and reviewability.
- Aligns with the collaborative git workflow grading criterion.

## 12. Final Functional Summary

The app now supports the following

- Strictly offline usage of bundled CBC topics and media.
- Granular favoriting for both topics and individual resources.
- External system viewing for all PDFs and Videos.
- Notes persistence and scheme builder.
- Scheme of Work creation, storage, and sharing.
- Optimized and prioritized navigation flow.

## 13. Suggested Future Enhancements

- Background scheduled sync with WorkManager
- Better source ranking for search links
- Scheme export to PDF directly inside app

- Role-based contributor/admin resource management